01 Secure File Vault & Multi-Algorithm Encryptor

01-secure-file-vault

Versioned authenticated file encryption with Argon2id, AES-256-GCM or ChaCha20-Poly1305, streaming chunks, bounded untrusted headers and atomic output.

Verified here: 266 passed; 0 failed; 0 skipped

Measured results

- 266/266 pytest cases passed, including 225 adversarial tamper/limit checks
- Argon2id default key derivation: 33.9 ms at t=3, 64 MiB, p=4 on this machine
- 64 MiB throughput: AES-GCM 1,585.9 MiB/s encrypt; ChaCha20-Poly1305 1,212.6 MiB/s
- Authenticated streaming with 64 KiB chunks; overhead 16,466 bytes per 64 MiB (0.025%)
- Hostile 4 GiB Argon2 header rejected before KDF; Streamlit AppTest clean

Data provenance and caveats

- Synthetic customer CSV and generated vault files from scripts/make_sample.py; no real personal data.
- Real local pytest, benchmark, and Streamlit AppTest runs on 2026-10-27.
- Benchmarks on one 16-CPU machine with OS page cache; laptop speed differs.
- Passphrase demo file holds only an obviously synthetic example, not a usable secret.
- Encryption does not protect against malware, keyloggers, plaintext before encryption, or file-name/size metadata.

Project contents

README.md, requirements.txt, app.py, src/securevault/__init__.py, src/securevault/__main__.py, src/securevault/cli.py, src/securevault/crypto.py, src/securevault/errors.py, src/securevault/fileops.py, src/securevault/format.py, src/securevault/kdf.py, src/securevault/memory_ops.py, tests/conftest.py, tests/test_app.py, tests/test_fileops_cli.py, tests/test_format.py, tests/test_roundtrip.py, tests/test_tamper.py, .gitignore, pyproject.toml, scripts/benchmark.py, scripts/demo_cli.py, scripts/make_sample.py, vault.py

Secure File Vault & Multi-Algorithm Encryptor

A command-line and web tool that encrypts files with a passphrase using **Argon2id** key derivation and either **AES-256-GCM** or **ChaCha20-Poly1305**, in a small, versioned, fully authenticated file format. Any change to a vault file (a flipped bit, a truncated tail, swapped chunks or an edited header) is detected, and nothing is written in that case.

Built to show applied cryptography the way it should be done in practice: vetted primitives only (`cryptography` and `argon2-cffi`, no home-made ciphers), explicit threat model, defensive parsing of untrusted input, and tests that attack the format.

Results from this build

What	Result (measured on this machine, 16 CPUs, Python 3.11.16)
Test suite	**266 passed in 3.09s** (pytest, 5 test files)
Tamper cases rejected	225 parametrised tamper/limit tests: bit flips in every header byte, the commitment, and the first, middle and tag bytes of every chunk; truncation at 5 points; truncation to whole chunks; appended data; swapped chunks; cipher-id swap
Hostile header (Argon2 memory = 4 GiB, 1M iterations, 4 GiB chunks)	rejected **before** key derivation, in under 0.5 s (asserted in tests)
Argon2id cost, default t=3, 64 MiB, p=4	33.9 ms per derivation (159.0 ms at 256 MiB)
AES-256-GCM throughput (64 MiB, 64 KiB chunks)	1585.9 MiB/s encrypt, 1568.4 MiB/s decrypt
ChaCha20-Poly1305 throughput	1212.6 MiB/s encrypt, 1193.9 MiB/s decrypt
Size overhead	82-byte preamble + 16 bytes per 64 KiB chunk (16,466 bytes on a 64 MiB file, 0.025%)
Memory use	one chunk (64 KiB) + the Argon2 memory cost, whatever the file size
Streamlit UI	AppTest passes with no exceptions; screenshots in `docs/screenshots/`

The numbers are in `outputs/metrics.json` (written by `scripts/benchmark.py`). The full CLI transcript, including every failure case, is in `samples/cli_output.txt`. Argon2 timings depend on the CPU: a laptop will be slower, which is the point of the KDF.

How it works

```
passphrase ??Argon2id(salt, t, m, p)??? master key (32 B)
                                    ??HKDF-SHA256("aead key")??????? k_enc
                                    ??HKDF-SHA256("header commitment")? k_commit

file = header(50 B) ? HMAC-SHA256(k_commit, header) ? chunk_0 ? chunk_1 ? ? ? chunk_last
chunk_i = AEAD_k_enc(nonce = prefix(7) ? i(4) ? last_flag(1), plaintext_i, AAD = header)
```

Header (50 bytes, big-endian, clear text but authenticated): magic `SFVAULT\0`, version, cipher id, KDF id, flags, Argon2 time cost, memory (KiB), parallelism, salt length, nonce-prefix length, chunk size, 16-byte random salt, 7-byte random nonce prefix. The full byte layout is documented at the top of `src/securevault/format.py`.

Decryption checks, cheapest and safest first:
1. Parse the header and bounds-check every field: time cost 1-10, memory 8 MiB-1 GiB (default ceiling 512 MiB, adjustable with `--max-memory-mib`), parallelism 1-16, chunk size 1 KiB-16 MiB. A hostile file cannot make the program allocate 4 GiB or loop for minutes.
2. Run Argon2id with those bounded parameters.
3. Verify the header commitment HMAC (constant-time compare). A wrong passphrase or an edited header fails here, before any plaintext exists.
4. Decrypt chunk by chunk. Each chunk's tag covers its position (counter in the nonce) and

the header (AAD).
5. Only the final chunk carries `last_flag = 1`. Removing whole chunks, reordering them or appending bytes fails authentication (the STREAM construction, as used by `age` and Google Tink).

Security design decisions

Decision	Why
Argon2id, RFC 9106 parameters (t=3, 64 MiB, p=4 by default)	Memory-hard, so every passphrase guess costs RAM as well as CPU, which blunts GPU/ASIC brute force. Parameters are stored per file so they can be raised later without breaking old files.
Bounds on header parameters	The header is attacker-controlled input. Without limits, a crafted file could ask for 4 GiB of Argon2 memory: a denial of service.
Fresh 16-byte salt and 7-byte nonce prefix from `os.urandom` on every encryption	Every file gets its own key, and within a file the counter makes every nonce unique, so a (key, nonce) pair is never reused. Encrypting the same file twice gives different output (tested over 20 runs).
Two AEAD ciphers	AES-256-GCM is fastest with AES-NI hardware. ChaCha20-Poly1305 is constant-time in software and safer on CPUs without AES instructions.
HMAC key commitment over the header	AES-GCM and ChaCha20-Poly1305 are not key-committing on their own. The commitment rejects a wrong key immediately and binds the file to a single key.
HKDF domain separation	The AEAD key and the commitment key are independent, so no key is used for two purposes.
Header as AAD on every chunk	Defence in depth: the header is authenticated even if the commitment check were bypassed.
Chunked streaming (64 KiB)	Bounded memory for any file size, with authentication per chunk.
One error message for wrong passphrase or tampering	Does not tell an attacker which check failed.
Atomic output	Output goes to a 0600 temp file in the target directory, is `fsync`ed, then published with `os.link` (fails if the target appeared meanwhile, so no check-then-write race) or `os.replace` with `--force`. On any failure the temp file is deleted: **a failed decryption never leaves partial plaintext at the output path**, and with `--force` the old file is kept.
Overwrite refused unless `--force`	Checked before the passphrase prompt and again atomically at publish time.
Passphrase only through `getpass` (or `--passphrase-file` for scripts)	Command-line arguments are visible in `ps`, `/proc/<pid>/cmdline` and shell history. `allow_abbrev=False` is set on the parser because a test caught that argparse would otherwise accept `--passphrase X` as an abbreviation of `--passphrase-file`.
Web UI keeps no files	Uploads are processed in `BytesIO`. The passphrase field is masked and cleared on submit, and the result is offered once as a download.

Project structure

```
01-secure-file-vault/
??? README.md
??? requirements.txt / pyproject.toml   (pip install -e . gives a `vault` command)
??? vault.py                  run the CLI without installing
??? app.py                    Streamlit UI (upload ? encrypt/decrypt ? download)
??? .streamlit/config.toml      50 MB upload cap, XSRF protection on, telemetry off
??? src/securevault/
?   ??? format.py             binary format, header pack/parse, resource bounds
?   ??? kdf.py                Argon2id + HKDF key schedule
?   ??? crypto.py             streaming chunked AEAD encrypt/decrypt
?   ??? fileops.py            atomic writes, overwrite refusal, `info`
?   ??? memory_ops.py           in-memory helpers for the web UI
```

```
?   ??? cli.py                encrypt / decrypt / info
?   ??? errors.py
??? tests/                    266 pytest tests + golden test vectors
??? scripts/
?   ??? make_sample.py        generates the synthetic sample data
?   ??? demo_cli.py           runs the real CLI and records samples/cli_output.txt
?   ??? benchmark.py          KDF timing, throughput, test run ? outputs/metrics.json
??? samples/                  synthetic CSV, demo passphrase, CLI transcript, demo
vaults
??? docs/screenshots/         Streamlit UI screenshots
??? outputs/metrics.json
```

How to run

```bash
python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements.txt

# encrypt (prompts for the passphrase twice; it is never echoed)
python vault.py encrypt report.pdf                    # -> report.pdf.vault
(AES-256-GCM)
python vault.py encrypt report.pdf -c chacha -o report.chacha.vault
python vault.py encrypt big.iso --memory-mib 256      # stronger KDF

# inspect the header (no passphrase needed; shows what anyone holding the file can see)
python vault.py info report.pdf.vault

# decrypt
python vault.py decrypt report.pdf.vault              # -> report.pdf (refuses to
overwrite)
python vault.py decrypt report.pdf.vault -o out.pdf --force

# scripts/CI: passphrase from a chmod-600 file, never from an argument
python vault.py decrypt x.vault --passphrase-file ~/.secrets/vault-pass

# web UI
streamlit run app.py

# tests, demo transcript and benchmark
pytest -q
python scripts/make_sample.py && python scripts/demo_cli.py
python scripts/benchmark.py
```

Exit codes: `0` ok, `1` error, `2` authentication failed (wrong passphrase, tampered or truncated), `3` output exists, `4` invalid or hostile vault file.

Sample output (real run, from `samples/cli_output.txt`)

```
$ vault decrypt samples/demo_run/customers.aes.vault -o samples/demo_run/wrong.csv
--passphrase-file samples/demo_run/wrong-pass.txt
error: authentication failed - wrong passphrase, or the file has been modified. No output
was written.
[exit code 2]

$ vault decrypt samples/demo_run/tampered.vault -o samples/demo_run/tampered.csv
--passphrase-file samples/demo-passphrase.txt
error: authentication failed - file is truncated or has been modified. No output was
```

written.
[exit code 2]

$ vault decrypt samples/demo_run/hostile.vault -o samples/demo_run/hostile.csv
--passphrase-file samples/demo-passphrase.txt
error: invalid vault file - argon2 memory 4096 MiB exceeds the allowed 512 MiB (raise
--max-memory-mib if you trust this file)
[exit code 4]
```

All bundled data is synthetic. `samples/synthetic_customers.csv` is generated by `scripts/make_sample.py`, and the passphrases in `samples/` and `tests/` (`demo-only-correct-horse-battery`, `vector-passphrase-not-secret`, `test-passphrase-not-real`) exist only for the demo. Never reuse them.

## Tests

| File | Covers |
|---|---|
| `test_roundtrip.py` (22) | both ciphers × sizes 0, 1, chunk?1, chunk, chunk+1, 3 chunks, 3 chunks+17, 200 KB; empty file is one authenticated chunk; unique salt/nonce over 20 encryptions; Unicode passphrase; 5 MiB through 1,280 chunks |
| `test_tamper.py` (225) | wrong passphrase (no output produced); bit flips in every header byte, the commitment and every chunk region including tags; truncation; header-only files; appended bytes; swapped chunks; cipher swap; bad magic or version; hostile Argon2/chunk parameters rejected fast; caller memory ceiling |
| `test_fileops_cli.py` (15) | atomic output; no partial plaintext when only the last chunk is corrupt; failed `--force` decrypt keeps the old file; overwrite refused then forced; same input/output; no `--passphrase` option (abbreviation trap); CLI round trips; exit codes; passphrase mismatch or too short; a **real pseudo-terminal** run proving the typed passphrase is not echoed |
| `test_format.py` (2) | byte layout matches the spec; golden vectors from v1.0.0 still decrypt |
| `test_app.py` (2) | in-memory helpers; Streamlit AppTest |

## Limitations: what this does NOT protect against

Encryption protects data **at rest and in transit from people who do not have the passphrase**. It is not magic:

- **Malware, keyloggers, a compromised OS.** Anything that can read your keystrokes, screen or process memory gets the passphrase and the plaintext. No file-encryption tool can fix an infected machine.
- **Weak passphrases.** Argon2id makes each guess expensive, not impossible. A short or common passphrase will still fall to a dictionary attack. Use 4-6 random words or a password manager.
- **Metadata disclosure.** The file name, size (plaintext size can be computed from ciphertext size, as `vault info` shows), timestamps, the fact that you use this tool, the cipher and the KDF settings are all visible. Nothing hides that a file exists or roughly how big it is.
- **Plaintext left elsewhere.** The original file is not deleted or wiped. Secure deletion is unreliable on SSDs and journaling filesystems anyway. Editors, thumbnails, swap and backups may hold copies.
- **Memory hygiene.** Python cannot reliably zero memory. The passphrase and keys live in the process until garbage collected and could reach swap or a core dump.
- **Temporary plaintext during decryption.** Verified chunks are written to a 0600 temp file that is deleted if a later chunk fails. It is never published, but it touched the disk until deletion.
- **The web UI over a network.** Run it on localhost. Remotely it needs HTTPS, and the server operator could see everything.
```

- **No key rotation, sharing, recovery or forward secrecy.** Lose the passphrase and the data is gone.
- **Not audited.** This is a portfolio/learning project built on audited libraries. Use established tools such as `age`, GnuPG, VeraCrypt or your OS's disk encryption for real secrets.

Resume bullets

- Built a secure file-encryption tool in Python (Argon2id KDF + AES-256-GCM / ChaCha20-Poly1305) with a versioned, fully authenticated binary format: header commitment HMAC, header-as-AAD and STREAM-style chunk nonces that detect bit flips, truncation, reordering and appended data.
- Wrote 266 pytest tests, including 225 adversarial tamper cases and hostile-header tests proving resource-exhaustion inputs (4 GiB Argon2 memory requests) are rejected before key derivation; caught and fixed an argparse abbreviation flaw that would have let passphrases onto the command line.
- Built crash-safe file handling (0600 temp file, fsync, atomic link/replace) so failed decryptions never leave partial plaintext, with streaming encryption at about 1,585 MiB/s (AES-GCM) using constant memory.
- Shipped a CLI (getpass-only secrets, overwrite protection, distinct exit codes) and a Streamlit web UI that processes files in memory without storing plaintext, plus a written threat model covering malware, keyloggers and metadata leakage.

Secure File Vault & Multi-Algorithm Encryptor — ACTUAL SAMPLE OUTPUT

```
# 1. encrypt with AES-256-GCM (default Argon2id t=3, 64 MiB, p=4)
$ vault encrypt samples/synthetic_customers.csv -o samples/demo_run/customers.aes.vault --passphrase-file samples/demo-passphrase.txt
encrypted samples/synthetic_customers.csv -> samples/demo_run/customers.aes.vault
  cipher AES-256-GCM, Argon2id t=3 m=64MiB p=4, 14514 -> 14612 bytes in 0.04s
[exit code 0]

# 2. encrypt with ChaCha20-Poly1305
$ vault encrypt samples/synthetic_customers.csv -o samples/demo_run/customers.chacha.vault -c chacha --passphrase-file samples/demo-passphrase.txt
encrypted samples/synthetic_customers.csv -> samples/demo_run/customers.chacha.vault
  cipher ChaCha20-Poly1305, Argon2id t=3 m=64MiB p=4, 14514 -> 14612 bytes in 0.04s
[exit code 0]

# 3. inspect the header - no passphrase needed (this is the metadata anyone can see)
$ vault info samples/demo_run/customers.aes.vault
file                          samples/demo_run/customers.aes.vault
format_version                1
cipher                        AES-256-GCM
kdf                           Argon2id
argon2_time_cost              3
argon2_memory_mib             64.0
argon2_parallelism            4
chunk_size_bytes              65536
salt_hex                      52f08dc85504e31ca084d82a13af3982
nonce_prefix_hex              f7cb6a31212615
file_size_bytes               14612
chunks                        1
plaintext_size_bytes_if_intact  14514
note: everything above is visible to anyone holding the file (no passphrase needed)
[exit code 0]

# 4. decrypt
$ vault decrypt samples/demo_run/customers.aes.vault -o samples/demo_run/customers.decrypted.csv --passphrase-file samples/demo-passphrase.txt
decrypted samples/demo_run/customers.aes.vault -> samples/demo_run/customers.decrypted.csv (AES-256-GCM, 14514 bytes in 0.04s)
[exit code 0]

# 4b. byte-for-byte identical to the original: True

# 5. decrypt again to the same path - refused without --force
$ vault decrypt samples/demo_run/customers.chacha.vault -o samples/demo_run/customers.decrypted.csv --passphrase-file samples/demo-passphrase.txt
error: refusing to overwrite existing file: samples/demo_run/customers.decrypted.csv (use --force)
[exit code 3]

# 6. wrong passphrase
$ vault decrypt samples/demo_run/customers.aes.vault -o samples/demo_run/wrong.csv --passphrase-file samples/demo_run/wrong-pass.txt
error: authentication failed - wrong passphrase, or the file has been modified. No output was written.
[exit code 2]

# 6b. output file created after failure: False

# 7. one bit flipped in the ciphertext
$ vault decrypt samples/demo_run/tampered.vault -o samples/demo_run/tampered.csv --passphrase-file samples/demo-passphrase.txt
error: authentication failed - file is truncated or has been modified. No output was written.
[exit code 2]

# 8. last 40 bytes cut off
$ vault decrypt samples/demo_run/truncated.vault -o samples/demo_run/truncated.csv --passphrase-file samples/demo-passphrase.txt
```

```
error: authentication failed - file is truncated or has been modified. No output was written.
[exit code 2]

# 9. hostile header asking for 4 GiB of Argon2 memory - rejected before key derivation
$ vault decrypt samples/demo_run/hostile.vault -o samples/demo_run/hostile.csv --passphrase-file samples/demo-passphrase.txt
error: invalid vault file - argon2 memory 4096 MiB exceeds the allowed 512 MiB (raise --max-memory-mib if you trust this file)
[exit code 4]

# 10. empty file
$ vault encrypt samples/demo_run/empty.txt --passphrase-file samples/demo-passphrase.txt
encrypted samples/demo_run/empty.txt -> samples/demo_run/empty.txt.vault
  cipher AES-256-GCM, Argon2id t=3 m=64MiB p=4, 0 -> 98 bytes in 0.04s
[exit code 0]

# 10b. decrypt empty file
$ vault decrypt samples/demo_run/empty.txt.vault -o samples/demo_run/empty.out --passphrase-file samples/demo-passphrase.txt
decrypted samples/demo_run/empty.txt.vault -> samples/demo_run/empty.out (AES-256-GCM, 0 bytes in 0.03s)
[exit code 0]

# 11. interactive getpass in a real terminal (typed characters are not echoed)
$ vault encrypt samples/synthetic_customers.csv -o samples/demo_run/interactive.vault
Passphrase:
Confirm passphrase:
encrypted samples/synthetic_customers.csv -> samples/demo_run/interactive.vault
  cipher AES-256-GCM, Argon2id t=3 m=64MiB p=4, 14514 -> 14612 bytes in 0.04s
[exit code 0]

```

01-secure-file-vault/requirements.txt

```
cryptography>=42.0
argon2-cffi>=23.1
streamlit>=1.43   # optional web UI (download_button on_click="ignore" needs 1.43+)
pytest>=8.0       # tests
pexpect>=4.9      # tests: drives the real getpass prompt through a pseudo-terminal (Linux/macOS)

```

01-secure-file-vault/app.py

```
"""Streamlit UI for the secure file vault.

    streamlit run app.py

Design rules:
  * uploads are processed in memory (BytesIO) and never written to disk by this app;
  * the passphrase field is type="password" inside a form with clear_on_submit,
    is never echoed, logged, or stored in st.session_state after the run;
  * results are offered once as a download; nothing is kept between sessions.
Run it on localhost. Over a network, use HTTPS - otherwise the passphrase and
file travel in clear text to the server.
"""
import sys
from pathlib import Path

import streamlit as st

sys.path.insert(0, str(Path(__file__).resolve().parent / "src"))
from securevault import AuthenticationError, Cipher, FormatError, VaultError  # noqa: E402
from securevault.memory_ops import MAX_UI_BYTES, decrypt_bytes, encrypt_bytes  # noqa: E402

st.set_page_config(page_title="Secure File Vault", page_icon="□", layout="centered")
st.title("□ Secure File Vault")
st.caption("Argon2id key derivation · AES-256-GCM or ChaCha20-Poly1305 · authenticated header · "
           "files are processed in memory and never stored")

mode = st.radio("What do you want to do?", ["Encrypt", "Decrypt"], horizontal=True)

with st.form("vault_form", clear_on_submit=True):
    upload = st.file_uploader("File", type=None, key="upload",
                              help=f"Up to {MAX_UI_BYTES // 2**20} MiB in the web UI; use the CLI
for bigger files")
    cipher_label = None
    if mode == "Encrypt":
        cipher_label = st.selectbox("Cipher", ["AES-256-GCM", "ChaCha20-Poly1305"])
    passphrase = st.text_input("Passphrase", type="password", autocomplete="off")
    confirm = st.text_input("Confirm passphrase", type="password", autocomplete="off") if mode ==
"Encrypt" else None
    submitted = st.form_submit_button(mode)

if submitted:
    if upload is None:
        st.error("Choose a file first.")
    elif not passphrase:
        st.error("Enter a passphrase.")
    elif mode == "Encrypt" and passphrase != confirm:
        st.error("Passphrases do not match.")
    elif mode == "Encrypt" and len(passphrase) < 8:
        st.error("Use at least 8 characters - a few random words is better.")
    else:
        data = upload.getvalue()
        try:
            with st.spinner("Deriving key with Argon2id and processing..."):
                if mode == "Encrypt":
                    cipher = Cipher.AES_256_GCM if cipher_label == "AES-256-GCM" else Cipher.CHACH
A20_POLY1305
                    result, name = encrypt_bytes(data, passphrase, cipher), upload.name + ".vault"
                else:
                    result = decrypt_bytes(data, passphrase)
                    name = upload.name[:-6] if upload.name.endswith(".vault") else upload.name + "
.decrypted"
            st.success(f"{mode}ed {len(data):,} -> {len(result):,} bytes.")
            st.download_button(f"Download {name}", data=result, file_name=name,
                               mime="application/octet-stream", on_click="ignore")
        except AuthenticationError:
```

```
            st.error("Wrong passphrase, or the file was modified or truncated. Nothing was produced.")
        except FormatError as exc:
            st.error(f"Not a valid vault file: {exc}")
        except VaultError as exc:
            st.error(str(exc))
        finally:
            passphrase = confirm = None      # drop our references (Python cannot wipe memory)

with st.expander("What this does and does not protect against"):
    st.markdown(
        "- **Protects** a file at rest or in transit against anyone without the passphrase, "
        "and detects any modification or truncation.\n"
        "- **Does not protect** against malware or a keylogger on this computer, a weak passphrase, "
        "or metadata: file size, timestamps and the header (cipher, KDF settings) stay visible.\n"
        "- Not independently audited - a learning/portfolio project.")

```

01-secure-file-vault/src/securevault/__init__.py

```
"""securevault - a small, auditable encrypted file vault.

Public API:
    encrypt_stream / decrypt_stream   - work on binary file-like objects
    encrypt_file / decrypt_file       - path based, atomic, refuse overwrite
    read_header                       - parse the (unencrypted) header for `info`
"""
from .errors import (VaultError, AuthenticationError, FormatError,
                     LimitExceededError, OutputExistsError)
from .format import (Cipher, Header, KdfParams, MAGIC, VERSION, DEFAULT_KDF,
                     DEFAULT_CHUNK_SIZE, read_header)
from .crypto import encrypt_stream, decrypt_stream
from .fileops import encrypt_file, decrypt_file

__version__ = "1.0.0"
__all__ = [
    "VaultError", "AuthenticationError", "FormatError", "LimitExceededError",
    "OutputExistsError", "Cipher", "Header", "KdfParams", "MAGIC", "VERSION",
    "DEFAULT_KDF", "DEFAULT_CHUNK_SIZE", "read_header", "encrypt_stream",
    "decrypt_stream", "encrypt_file", "decrypt_file", "__version__",
]

```

01-secure-file-vault/src/securevault/__main__.py

```
import sys
from .cli import main

sys.exit(main())

```

```
"""Command line interface:  vault encrypt | decrypt | info

The passphrase is NEVER accepted as a command-line argument (it would land in
shell history and be visible to other users via `ps` / /proc/<pid>/cmdline).
It is read with getpass from the terminal, or for automation from a file given
by --passphrase-file (first line; keep that file chmod 600).
"""
from __future__ import annotations

import argparse
import getpass
import json
import os
import stat
import sys
import time
from pathlib import Path

from . import __version__
from .errors import (AuthenticationError, FormatError, LimitExceededError,
                     OutputExistsError, VaultError)
from .fileops import decrypt_file, describe, encrypt_file
from .format import (DEFAULT_CHUNK_SIZE, DEFAULT_DECRYPT_MAX_MEMORY_KIB, DEFAULT_KDF,
                     Cipher, KdfParams)

EXIT_OK, EXIT_ERROR, EXIT_AUTH, EXIT_EXISTS, EXIT_FORMAT = 0, 1, 2, 3, 4
MIN_PASSPHRASE_LEN = 8


def _read_passphrase(args, confirm: bool) -> str:
    if args.passphrase_file:
        p = Path(args.passphrase_file)
        mode = p.stat().st_mode
        if mode & (stat.S_IRWXG | stat.S_IRWXO):
            print(f"warning: {p} is readable by other users; chmod 600 it", file=sys.stderr)
        line = p.read_text(encoding="utf-8").splitlines()
        return line[0] if line else ""
    pw = getpass.getpass("Passphrase: ")
    if confirm:
        again = getpass.getpass("Confirm passphrase: ")
        if pw != again:
            raise VaultError("passphrases do not match")
    return pw


def _default_out(src: str, encrypting: bool) -> str:
    if encrypting:
        return src + ".vault"
    return src[:-6] if src.endswith(".vault") else src + ".decrypted"


def cmd_encrypt(args) -> int:
    out = args.output or _default_out(args.input, True)
    if os.path.exists(out) and not args.force:      # fail fast, before asking for a passphrase
        raise OutputExistsError(f"refusing to overwrite existing file: {out} (use --force)")
    kdf = KdfParams(args.time_cost, args.memory_mib * 1024, args.parallelism)
    kdf.validate()
    pw = _read_passphrase(args, confirm=True)
    if len(pw) < MIN_PASSPHRASE_LEN:
        raise VaultError(f"passphrase must be at least {MIN_PASSPHRASE_LEN} characters")
    t0 = time.perf_counter()
    header = encrypt_file(args.input, out, pw, force=args.force,
                          cipher=Cipher.from_name(args.cipher), kdf=kdf,
                          chunk_size=args.chunk_kib * 1024)
    dt = time.perf_counter() - t0
```

```
    print(f"encrypted {args.input} -> {out}")
    print(f"  cipher {header.cipher.label}, Argon2id t={kdf.time_cost} "
          f"m={kdf.memory_kib // 1024}MiB p={kdf.parallelism}, "
          f"{os.path.getsize(args.input)} -> {os.path.getsize(out)} bytes in {dt:.2f}s")
    return EXIT_OK


def cmd_decrypt(args) -> int:
    out = args.output or _default_out(args.input, False)
    if os.path.exists(out) and not args.force:
        raise OutputExistsError(f"refusing to overwrite existing file: {out} (use --force)")
    pw = _read_passphrase(args, confirm=False)
    t0 = time.perf_counter()
    header = decrypt_file(args.input, out, pw, force=args.force,
                          max_memory_kib=args.max_memory_mib * 1024)
    dt = time.perf_counter() - t0
    print(f"decrypted {args.input} -> {out} ({header.cipher.label}, "
          f"{os.path.getsize(out)} bytes in {dt:.2f}s)")
    return EXIT_OK


def cmd_info(args) -> int:
    facts = describe(args.input)
    if args.json:
        print(json.dumps(facts, indent=2))
    else:
        width = max(len(k) for k in facts)
        for k, v in facts.items():
            print(f"{k:<{width}}  {v}")
        print("note: everything above is visible to anyone holding the file (no passphrase needed)
")
    return EXIT_OK


def build_parser() -> argparse.ArgumentParser:
    # allow_abbrev=False: otherwise argparse would accept "--passphrase X" as an
    # abbreviation of --passphrase-file, inviting secrets onto the command line.
    p = argparse.ArgumentParser(prog="vault", allow_abbrev=False, description="Secure file vault (
Argon2id + AES-256-GCM / ChaCha20-Poly1305)")
    p.add_argument("--version", action="version", version=f"%(prog)s {__version__}")
    sub = p.add_subparsers(dest="command", required=True)

    def common(sp):
        sp.add_argument("input")
        sp.add_argument("-o", "--output", help="output path (default: input + .vault / minus .vaul
t)")
        sp.add_argument("-f", "--force", action="store_true", help="overwrite the output if it exi
sts")
        sp.add_argument("--passphrase-file", help="read passphrase from first line of this file (f
or scripts)")

    e = sub.add_parser("encrypt", allow_abbrev=False, help="encrypt a file")
    common(e)
    e.add_argument("-c", "--cipher", default="aes", choices=["aes", "chacha"])
    e.add_argument("--time-cost", type=int, default=DEFAULT_KDF.time_cost)
    e.add_argument("--memory-mib", type=int, default=DEFAULT_KDF.memory_kib // 1024)
    e.add_argument("--parallelism", type=int, default=DEFAULT_KDF.parallelism)
    e.add_argument("--chunk-kib", type=int, default=DEFAULT_CHUNK_SIZE // 1024)
    e.set_defaults(func=cmd_encrypt)

    d = sub.add_parser("decrypt", allow_abbrev=False, help="decrypt a file")
    common(d)
    d.add_argument("--max-memory-mib", type=int, default=DEFAULT_DECRYPT_MAX_MEMORY_KIB // 1024,
                   help="refuse files whose Argon2 memory cost is above this (default 512)")
    d.set_defaults(func=cmd_decrypt)
```

```
126
127     i = sub.add_parser("info", allow_abbrev=False, help="show the unencrypted header (no passphras
    e needed)")
128     i.add_argument("input")
129     i.add_argument("--json", action="store_true")
130     i.set_defaults(func=cmd_info)
131     return p
132
133
134 def main(argv=None) -> int:
135     args = build_parser().parse_args(argv)
136     try:
137         return args.func(args)
138     except AuthenticationError as exc:
139         print(f"error: authentication failed - {exc}. No output was written.", file=sys.stderr)
140         return EXIT_AUTH
141     except OutputExistsError as exc:
142         print(f"error: {exc}", file=sys.stderr)
143         return EXIT_EXISTS
144     except (FormatError, LimitExceededError) as exc:
145         print(f"error: invalid vault file - {exc}", file=sys.stderr)
146         return EXIT_FORMAT
147     except (VaultError, ValueError, OSError) as exc:
148         print(f"error: {exc}", file=sys.stderr)
149         return EXIT_ERROR
150     except KeyboardInterrupt:
151         print("\ninterrupted; no output written", file=sys.stderr)
152         return 130
153
154
155 if __name__ == "__main__":
156     sys.exit(main())
157
```

```
"""Streaming authenticated encryption (chunked AEAD, STREAM construction).

Memory use is bounded: one chunk (default 64 KiB) of plaintext + ciphertext is
held at a time, plus the Argon2id memory cost. File size is not limited by RAM.
"""
from __future__ import annotations

import hmac
import hashlib
import os
import struct
from typing import BinaryIO, Callable

from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM, ChaCha20Poly1305

from .errors import AuthenticationError
from .format import (DEFAULT_CHUNK_SIZE, DEFAULT_DECRYPT_MAX_MEMORY_KIB, DEFAULT_KDF,
                     MAX_CHUNK, MIN_CHUNK, NONCE_PREFIX_LEN, SALT_LEN, TAG_LEN,
                     Cipher, Header, KdfParams, read_exact, read_header)
from .kdf import derive_keys

MAX_CHUNKS = 2 ** 32 - 1          # 32-bit counter in the nonce

ProgressFn = Callable[[int], None]


def _aead(cipher: Cipher, key: bytes):
    return AESGCM(key) if cipher is Cipher.AES_256_GCM else ChaCha20Poly1305(key)


def _nonce(prefix: bytes, counter: int, last: bool) -> bytes:
    return prefix + struct.pack(">I", counter) + (b"\x01" if last else b"\x00")


def _commitment(k_commit: bytes, raw_header: bytes) -> bytes:
    return hmac.new(k_commit, raw_header, hashlib.sha256).digest()


def encrypt_stream(src: BinaryIO, dst: BinaryIO, passphrase: str | bytes, *,
                   cipher: Cipher = Cipher.AES_256_GCM, kdf: KdfParams = DEFAULT_KDF,
                   chunk_size: int = DEFAULT_CHUNK_SIZE,
                   progress: ProgressFn | None = None) -> Header:
    """Encrypt everything readable from `src` into `dst`. Returns the header used.

    A fresh random 16-byte salt and 7-byte nonce prefix come from os.urandom
    for every call, so the same file + passphrase never produces the same
    output and no (key, nonce) pair is ever reused.
    """
    kdf.validate()
    if not MIN_CHUNK <= chunk_size <= MAX_CHUNK:
        raise ValueError(f"chunk_size must be in [{MIN_CHUNK}, {MAX_CHUNK}]")
    header = Header(cipher=Cipher(cipher), kdf=kdf, chunk_size=chunk_size,
                    salt=os.urandom(SALT_LEN), nonce_prefix=os.urandom(NONCE_PREFIX_LEN))
    raw = header.pack()
    k_enc, k_commit = derive_keys(passphrase, header.salt, kdf)
    aead = _aead(header.cipher, k_enc)
    dst.write(raw)
    dst.write(_commitment(k_commit, raw))

    # One chunk of look-ahead tells us whether the current chunk is the last.
    current = read_exact(src, chunk_size)
    counter = 0
    while True:
        nxt = read_exact(src, chunk_size) if len(current) == chunk_size else b""
```

```
        last = not nxt
        if counter > MAX_CHUNKS:
            raise ValueError("input too large for 32-bit chunk counter")
        dst.write(aead.encrypt(_nonce(header.nonce_prefix, counter, last), current, raw))
        if progress:
            progress(len(current))
        if last:
            break
        current, counter = nxt, counter + 1
    return header


def decrypt_stream(src: BinaryIO, dst: BinaryIO, passphrase: str | bytes, *,
                   max_memory_kib: int = DEFAULT_DECRYPT_MAX_MEMORY_KIB,
                   progress: ProgressFn | None = None) -> Header:
    """Decrypt `src` into `dst`, authenticating every chunk before writing it.

    Order of checks (cheap and safe first):
      1. header parse + resource bounds  -> FormatError / LimitExceededError
      2. Argon2id with the bounded params
      3. key commitment HMAC             -> AuthenticationError (wrong passphrase
                                            or header edited) before any output
      4. each chunk's AEAD tag           -> AuthenticationError
      5. exactly one final chunk, nothing after it -> AuthenticationError

    NOTE: plaintext of already-verified chunks has been written to `dst` when a
    later chunk fails. File-level callers (fileops.decrypt_file) write into a
    temporary file and delete it on failure so no partial plaintext is published.
    """
    header, raw, commitment = read_header(src, max_memory_kib)
    k_enc, k_commit = derive_keys(passphrase, header.salt, header.kdf)
    if not hmac.compare_digest(commitment, _commitment(k_commit, raw)):
        raise AuthenticationError("wrong passphrase, or the file has been modified")
    aead = _aead(header.cipher, k_enc)
    block = header.chunk_size + TAG_LEN

    current = read_exact(src, block)
    counter = 0
    while True:
        if len(current) < TAG_LEN:
            raise AuthenticationError("file is truncated or has been modified")
        # Peek one byte: a full block followed by EOF is a full-size final chunk.
        nxt = read_exact(src, block) if len(current) == block else b""
        last = not nxt
        try:
            plain = aead.decrypt(_nonce(header.nonce_prefix, counter, last), current, raw)
        except InvalidTag:
            raise AuthenticationError("file is truncated or has been modified") from None
        dst.write(plain)
        if progress:
            progress(len(plain))
        if last:
            return header
        if counter >= MAX_CHUNKS:
            raise AuthenticationError("too many chunks")
        current, counter = nxt, counter + 1

```

01-secure-file-vault/src/securevault/errors.py

```
"""Exception hierarchy. Every error the vault raises is a VaultError."""


class VaultError(Exception):
    """Base class for all vault errors."""


class FormatError(VaultError):
    """The input is not a vault file, uses an unknown version, or is malformed."""


class LimitExceededError(FormatError):
    """A header field asks for more resources than this program allows.

    Raised *before* any key derivation, so a hostile header cannot make us
    allocate gigabytes of Argon2 memory or spin for minutes.
    """


class AuthenticationError(VaultError):
    """Wrong passphrase, or the file was modified/truncated.

    These cases are deliberately indistinguishable: telling an attacker *which*
    check failed only helps them.
    """


class OutputExistsError(VaultError):
    """The output path already exists and overwrite was not requested."""

```

01-secure-file-vault/src/securevault/fileops.py

```
"""Path-level operations: atomic output, overwrite refusal, no partial plaintext."""
from __future__ import annotations

import contextlib
import os
import tempfile
from pathlib import Path
from typing import BinaryIO, Callable

from .crypto import decrypt_stream, encrypt_stream
from .errors import OutputExistsError, VaultError
from .format import DEFAULT_DECRYPT_MAX_MEMORY_KIB, Header, read_header


def _check_paths(src: Path, dst: Path, force: bool) -> None:
    if not src.is_file():
        raise VaultError(f"input is not a regular file: {src}")
    if dst.exists():
        with contextlib.suppress(OSError):
            if os.path.samefile(src, dst):
                raise VaultError("input and output are the same file")
        if not force:
            raise OutputExistsError(f"refusing to overwrite existing file: {dst} (use --force)")
        if dst.is_dir():
            raise VaultError(f"output is a directory: {dst}")


def atomic_write(dst: Path, writer: Callable[[BinaryIO], object], force: bool = False):
    """Run writer(fileobj) into a temp file next to `dst`, then publish atomically.

    * The temp file is created with mode 0600 in the destination directory (so
      the final rename stays on one filesystem and is atomic).
    * On ANY exception the temp file is deleted and `dst` is left untouched, so
      a failed decryption never leaves partial plaintext at the output path.
    * Without force, publication uses os.link(), which fails if `dst` appeared
      in the meantime - no check-then-write race. With force, os.replace().
    """
    dst = Path(dst)
    directory = dst.parent if str(dst.parent) else Path(".")
    fd, tmp_name = tempfile.mkstemp(prefix=f".{dst.name}.", suffix=".part", dir=directory)
    tmp = Path(tmp_name)
    try:
        os.fchmod(fd, 0o600) if hasattr(os, "fchmod") else None
        with os.fdopen(fd, "wb") as fh:
            result = writer(fh)
            fh.flush()
            os.fsync(fh.fileno())
        if force:
            os.replace(tmp, dst)
        else:
            try:
                os.link(tmp, dst)               # atomic "create if absent"
            except FileExistsError:
                raise OutputExistsError(f"refusing to overwrite existing file: {dst}") from None
            except OSError:
                # Filesystem without hard links (e.g. FAT/exFAT): best effort.
                if dst.exists():
                    raise OutputExistsError(f"refusing to overwrite existing file: {dst}") from No
ne
                os.replace(tmp, dst)
            with contextlib.suppress(FileNotFoundError):
                tmp.unlink()
        with contextlib.suppress(OSError):    # persist the directory entry
            dfd = os.open(directory, os.O_RDONLY)
            try:
```

```
                os.fsync(dfd)
            finally:
                os.close(dfd)
        return result
    except BaseException:
        with contextlib.suppress(FileNotFoundError):
            tmp.unlink()
        raise


def encrypt_file(src, dst, passphrase, *, force: bool = False, **kwargs) -> Header:
    src, dst = Path(src), Path(dst)
    _check_paths(src, dst, force)
    with open(src, "rb") as fin:
        return atomic_write(dst, lambda fout: encrypt_stream(fin, fout, passphrase, **kwargs), for
ce)


def decrypt_file(src, dst, passphrase, *, force: bool = False,
                 max_memory_kib: int = DEFAULT_DECRYPT_MAX_MEMORY_KIB, **kwargs) -> Header:
    src, dst = Path(src), Path(dst)
    _check_paths(src, dst, force)
    with open(src, "rb") as fin:
        # Parse + bounds-check the header before creating any output file.
        read_header(fin, max_memory_kib)
        fin.seek(0)
        return atomic_write(
            dst, lambda fout: decrypt_stream(fin, fout, passphrase,
                                             max_memory_kib=max_memory_kib, **kwargs), force)


def describe(path) -> dict:
    """Header facts for `vault info` - readable WITHOUT the passphrase.

    This is exactly the metadata an attacker holding the file also sees.
    """
    path = Path(path)
    size = path.stat().st_size
    with open(path, "rb") as fh:
        header, _raw, _commit = read_header(fh, max_memory_kib=None)
    from .format import PREAMBLE_LEN, TAG_LEN
    body = size - PREAMBLE_LEN
    block = header.chunk_size + TAG_LEN
    n_chunks = max(1, -(-body // block)) if body > 0 else 0
    plaintext = body - n_chunks * TAG_LEN
    return {
        "file": str(path),
        "format_version": header.version,
        "cipher": header.cipher.label,
        "kdf": "Argon2id",
        "argon2_time_cost": header.kdf.time_cost,
        "argon2_memory_mib": header.kdf.memory_kib / 1024,
        "argon2_parallelism": header.kdf.parallelism,
        "chunk_size_bytes": header.chunk_size,
        "salt_hex": header.salt.hex(),
        "nonce_prefix_hex": header.nonce_prefix.hex(),
        "file_size_bytes": size,
        "chunks": n_chunks,
        "plaintext_size_bytes_if_intact": plaintext,
    }

```

```
"""Binary container format, version 1.

All integers are big-endian. The 50-byte header is stored in clear text (the
decryptor needs the salt and KDF parameters before it has a key) but it is
*authenticated* twice:
  1. a 32-byte HMAC-SHA256 "key commitment" over the header follows it, and
  2. the header bytes are the Associated Data (AAD) of every AEAD chunk.
Changing any header byte therefore makes decryption fail.

    offset size field
    ------ ---- ------------------------------------------------------------
      0     8   magic            b"SFVAULT\x00"
      8     1   version          1
      9     1   cipher id        1 = AES-256-GCM, 2 = ChaCha20-Poly1305
     10     1   kdf id           1 = Argon2id
     11     1   flags            reserved, must be 0
     12     4   argon2 time_cost (iterations)
     16     4   argon2 memory    (KiB)
     20     1   argon2 parallelism (lanes)
     21     1   salt length      must be 16
     22     1   nonce prefix len must be 7
     23     4   chunk size       plaintext bytes per chunk
     27    16   salt             random per encryption
     43     7   nonce prefix     random per encryption
    ------ ---- ------------------------------------------------------------
     50    32   commitment       HMAC-SHA256(k_commit, header[0:50])
     82     ..  chunks           each = ciphertext(len <= chunk size) || 16-byte tag

Chunk nonce (12 bytes) = nonce_prefix(7) || counter(uint32) || last_flag(1).
The final chunk is the only one with last_flag = 1, so dropping trailing chunks,
reordering chunks or appending data is detected (the STREAM construction of
Hoang, Reyhanitabar, Rogaway & Vizar, 2015, also used by age and Tink).
"""
from __future__ import annotations

import enum
import struct
from dataclasses import dataclass
from typing import BinaryIO

from .errors import AuthenticationError, FormatError, LimitExceededError

MAGIC = b"SFVAULT\x00"
VERSION = 1
KDF_ARGON2ID = 1
SALT_LEN = 16
NONCE_PREFIX_LEN = 7
TAG_LEN = 16
COMMIT_LEN = 32

_HEADER_STRUCT = struct.Struct(">8sBBBBIIBBBI")   # fixed fields, 27 bytes
HEADER_LEN = _HEADER_STRUCT.size + SALT_LEN + NONCE_PREFIX_LEN   # 50
PREAMBLE_LEN = HEADER_LEN + COMMIT_LEN                           # 82

# ---- resource bounds -------------------------------------------------------
# Hard limits accepted from ANY header (anti resource-exhaustion). A caller may
# lower the memory ceiling further with max_memory_kib.
MIN_TIME_COST, MAX_TIME_COST = 1, 10
MIN_MEMORY_KIB, MAX_MEMORY_KIB = 8 * 1024, 1024 * 1024     # 8 MiB .. 1 GiB
MIN_PARALLELISM, MAX_PARALLELISM = 1, 16
MIN_CHUNK, MAX_CHUNK = 1024, 16 * 1024 * 1024              # 1 KiB .. 16 MiB

DEFAULT_CHUNK_SIZE = 64 * 1024
DEFAULT_DECRYPT_MAX_MEMORY_KIB = 512 * 1024                # 512 MiB

```

```
66
67 class Cipher(enum.IntEnum):
68     AES_256_GCM = 1
69     CHACHA20_POLY1305 = 2
70
71     @classmethod
72     def from_name(cls, name: str) -> "Cipher":
73         aliases = {"aes": cls.AES_256_GCM, "aes-256-gcm": cls.AES_256_GCM,
74                    "aesgcm": cls.AES_256_GCM, "chacha": cls.CHACHA20_POLY1305,
75                    "chacha20": cls.CHACHA20_POLY1305,
76                    "chacha20-poly1305": cls.CHACHA20_POLY1305}
77         try:
78             return aliases[name.lower()]
79         except KeyError:
80             raise ValueError(f"unknown cipher {name!r}; use 'aes' or 'chacha'") from None
81
82     @property
83     def label(self) -> str:
84         return {1: "AES-256-GCM", 2: "ChaCha20-Poly1305"}[int(self)]
85
86
87 @dataclass(frozen=True)
88 class KdfParams:
89     """Argon2id cost parameters (RFC 9106)."""
90     time_cost: int = 3
91     memory_kib: int = 64 * 1024        # 64 MiB, RFC 9106 "second recommended option"
92     parallelism: int = 4
93
94     def validate(self, max_memory_kib: int | None = None) -> None:
95         """Raise LimitExceededError if outside the hard (or caller) bounds."""
96         if not MIN_TIME_COST <= self.time_cost <= MAX_TIME_COST:
97             raise LimitExceededError(
98                 f"argon2 time_cost {self.time_cost} outside [{MIN_TIME_COST}, {MAX_TIME_COST}]")
99         if not MIN_PARALLELISM <= self.parallelism <= MAX_PARALLELISM:
100             raise LimitExceededError(
101                 f"argon2 parallelism {self.parallelism} outside [{MIN_PARALLELISM}, {MAX_PARALLELI
    SM}]")
102         if self.memory_kib < max(MIN_MEMORY_KIB, 8 * self.parallelism):
103             raise LimitExceededError(f"argon2 memory {self.memory_kib} KiB below minimum {MIN_MEMO
    RY_KIB} KiB")
104         ceiling = MAX_MEMORY_KIB if max_memory_kib is None else min(MAX_MEMORY_KIB, max_memory_kib
    )
105         if self.memory_kib > ceiling:
106             raise LimitExceededError(
107                 f"argon2 memory {self.memory_kib // 1024} MiB exceeds the allowed "
108                 f"{ceiling // 1024} MiB (raise --max-memory-mib if you trust this file)")
109
110
111 DEFAULT_KDF = KdfParams()
112
113
114 @dataclass(frozen=True)
115 class Header:
116     cipher: Cipher
117     kdf: KdfParams
118     chunk_size: int
119     salt: bytes
120     nonce_prefix: bytes
121     version: int = VERSION
122     flags: int = 0
123
124     def pack(self) -> bytes:
125         fixed = _HEADER_STRUCT.pack(MAGIC, self.version, int(self.cipher), KDF_ARGON2ID,
126                                     self.flags, self.kdf.time_cost, self.kdf.memory_kib,
127                                     self.kdf.parallelism, len(self.salt),
```

```
                                    len(self.nonce_prefix), self.chunk_size)
        out = fixed + self.salt + self.nonce_prefix
        assert len(out) == HEADER_LEN
        return out

    @classmethod
    def unpack(cls, data: bytes, max_memory_kib: int | None = None) -> "Header":
        """Parse and bounds-check a header. Never trusts a field before checking it."""
        if len(data) < HEADER_LEN:
            raise FormatError("file too short to be a vault file")
        (magic, version, cipher_id, kdf_id, flags, t, m, p, salt_len, np_len,
         chunk) = _HEADER_STRUCT.unpack_from(data, 0)
        if magic != MAGIC:
            raise FormatError("not a vault file (bad magic bytes)")
        if version != VERSION:
            raise FormatError(f"unsupported format version {version} (this build reads {VERSION})"
)
        try:
            cipher = Cipher(cipher_id)
        except ValueError:
            raise FormatError(f"unknown cipher id {cipher_id}") from None
        if kdf_id != KDF_ARGON2ID:
            raise FormatError(f"unknown KDF id {kdf_id}")
        if flags != 0:
            raise FormatError(f"unknown flags 0x{flags:02x}")
        if salt_len != SALT_LEN or np_len != NONCE_PREFIX_LEN:
            raise FormatError("unexpected salt/nonce length")
        if not MIN_CHUNK <= chunk <= MAX_CHUNK:
            raise LimitExceededError(f"chunk size {chunk} outside [{MIN_CHUNK}, {MAX_CHUNK}]")
        kdf = KdfParams(t, m, p)
        kdf.validate(max_memory_kib)
        off = _HEADER_STRUCT.size
        return cls(cipher=cipher, kdf=kdf, chunk_size=chunk,
                   salt=bytes(data[off:off + SALT_LEN]),
                   nonce_prefix=bytes(data[off + SALT_LEN:HEADER_LEN]),
                   version=version, flags=flags)


def read_exact(src: BinaryIO, n: int) -> bytes:
    """Read up to n bytes, looping over short reads; returns fewer only at EOF."""
    buf = bytearray()
    while len(buf) < n:
        part = src.read(n - len(buf))
        if not part:
            break
        buf += part
    return bytes(buf)


def read_header(src: BinaryIO, max_memory_kib: int | None = None) -> tuple[Header, bytes, bytes]:
    """Read header + commitment from a stream. Returns (header, raw_header, commitment)."""
    pre = read_exact(src, PREAMBLE_LEN)
    if len(pre) < HEADER_LEN:
        raise FormatError("file too short to be a vault file")
    header = Header.unpack(pre[:HEADER_LEN], max_memory_kib)
    if len(pre) < PREAMBLE_LEN:
        raise AuthenticationError("file is truncated or has been modified")
    return header, pre[:HEADER_LEN], pre[HEADER_LEN:]
```

01-secure-file-vault/src/securevault/kdf.py

```
"""Key derivation: passphrase --Argon2id--> master key --HKDF--> subkeys."""
from __future__ import annotations

from argon2.low_level import Type, hash_secret_raw
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.hkdf import HKDF

from .format import KdfParams

KEY_LEN = 32


def _hkdf(master: bytes, info: bytes) -> bytes:
    return HKDF(algorithm=hashes.SHA256(), length=KEY_LEN, salt=None, info=info).derive(master)


def derive_keys(passphrase: str | bytes, salt: bytes, params: KdfParams) -> tuple[bytes, bytes]:
    """Return (encryption_key, commitment_key).

    Argon2id makes each passphrase guess cost `memory_kib` of RAM and
    `time_cost` passes over it, which is what slows down offline brute force.
    HKDF then splits the master key into two independent subkeys so the
    commitment MAC and the AEAD never share a key.

    The caller MUST have validated `params` first (Header.unpack does).
    """
    if isinstance(passphrase, str):
        passphrase = passphrase.encode("utf-8")
    if not passphrase:
        raise ValueError("empty passphrase")
    master = hash_secret_raw(secret=passphrase, salt=salt, time_cost=params.time_cost,
                             memory_cost=params.memory_kib, parallelism=params.parallelism,
                             hash_len=KEY_LEN, type=Type.ID)
    return _hkdf(master, b"securevault v1 aead key"), _hkdf(master, b"securevault v1 header commit
ment")

```

01-secure-file-vault/src/securevault/memory_ops.py

```
"""In-memory helpers for the web UI: bytes in, bytes out, nothing touches disk."""
from __future__ import annotations

import io

from .crypto import decrypt_stream, encrypt_stream
from .errors import VaultError
from .format import DEFAULT_KDF, Cipher, KdfParams

MAX_UI_BYTES = 50 * 1024 * 1024          # matches .streamlit/config.toml maxUploadSize
UI_MAX_MEMORY_KIB = 256 * 1024           # hostile-header ceiling for uploaded vaults


def encrypt_bytes(data: bytes, passphrase: str, cipher: Cipher = Cipher.AES_256_GCM,
                  kdf: KdfParams = DEFAULT_KDF) -> bytes:
    if len(data) > MAX_UI_BYTES:
        raise VaultError(f"file larger than {MAX_UI_BYTES // 2**20} MiB; use the CLI")
    out = io.BytesIO()
    encrypt_stream(io.BytesIO(data), out, passphrase, cipher=cipher, kdf=kdf)
    return out.getvalue()


def decrypt_bytes(blob: bytes, passphrase: str) -> bytes:
    """Decrypt fully in memory; on failure the partial buffer is discarded."""
    if len(blob) > MAX_UI_BYTES + 2 * 2**20:
        raise VaultError("file too large for the web UI; use the CLI")
    out = io.BytesIO()
    try:
        decrypt_stream(io.BytesIO(blob), out, passphrase, max_memory_kib=UI_MAX_MEMORY_KIB)
    except BaseException:
        out.seek(0)
        out.truncate()                   # drop verified-but-partial plaintext
        raise
    return out.getvalue()

```

01-secure-file-vault/tests/conftest.py

```python
import sys
from pathlib import Path

import pytest

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))

from securevault import KdfParams  # noqa: E402

# Minimum allowed Argon2id cost keeps the suite fast; production default is 64 MiB.
FAST_KDF = KdfParams(time_cost=1, memory_kib=8 * 1024, parallelism=1)
PASS = "test-passphrase-not-real"   # synthetic, never a real password


@pytest.fixture
def fast_kdf():
    return FAST_KDF


@pytest.fixture
def passphrase():
    return PASS

```

01-secure-file-vault/tests/test_app.py

```python
"""Streamlit UI smoke test + the in-memory helpers it uses."""
from pathlib import Path

import pytest

from securevault import AuthenticationError, Cipher
from securevault.memory_ops import decrypt_bytes, encrypt_bytes

ROOT = Path(__file__).resolve().parents[1]


def test_memory_roundtrip(fast_kdf, passphrase):
    for c in Cipher:
        blob = encrypt_bytes(b"in-memory data", passphrase, c, fast_kdf)
        assert decrypt_bytes(blob, passphrase) == b"in-memory data"
        with pytest.raises(AuthenticationError):
            decrypt_bytes(blob, "wrong-passphrase")


def test_streamlit_app_runs():
    st_testing = pytest.importorskip("streamlit.testing.v1")
    at = st_testing.AppTest.from_file(str(ROOT / "app.py"), default_timeout=60).run()
    assert not at.exception
    # passphrase input exists and is a password field
    assert any(t.label == "Passphrase" for t in at.text_input)
    at.radio[0].set_value("Decrypt").run()
    assert not at.exception
    assert not any(t.label == "Confirm passphrase" for t in at.text_input)

```

01-secure-file-vault/tests/test_fileops_cli.py

```
"""Atomic output, overwrite protection and the CLI (including real getpass)."""
import os
import sys
from pathlib import Path

import pytest

from securevault import (AuthenticationError, OutputExistsError, decrypt_file,
                         encrypt_file)
from securevault.cli import (EXIT_AUTH, EXIT_EXISTS, EXIT_FORMAT, EXIT_OK,
                             build_parser, main)
from securevault.format import TAG_LEN

ROOT = Path(__file__).resolve().parents[1]
FAST = ["--time-cost", "1", "--memory-mib", "8", "--parallelism", "1"]


@pytest.fixture
def pwfile(tmp_path, passphrase):
    p = tmp_path / "pw.txt"
    p.write_text(passphrase + "\n")
    p.chmod(0o600)
    return str(p)


def _leftovers(d):
    return [p.name for p in Path(d).iterdir() if p.name.endswith(".part")]


# ---- library level ----------------------------------------------------------

def test_encrypt_decrypt_file(tmp_path, fast_kdf, passphrase):
    src = tmp_path / "doc.bin"
    src.write_bytes(os.urandom(100_000))
    encrypt_file(src, tmp_path / "doc.vault", passphrase, kdf=fast_kdf)
    decrypt_file(tmp_path / "doc.vault", tmp_path / "out.bin", passphrase)
    assert (tmp_path / "out.bin").read_bytes() == src.read_bytes()
    assert oct((tmp_path / "out.bin").stat().st_mode & 0o777) == "0o600"
    assert _leftovers(tmp_path) == []


def test_refuses_overwrite_without_force(tmp_path, fast_kdf, passphrase):
    src = tmp_path / "a.txt"
    src.write_text("secret")
    dst = tmp_path / "a.vault"
    dst.write_text("precious existing content")
    with pytest.raises(OutputExistsError):
        encrypt_file(src, dst, passphrase, kdf=fast_kdf)
    assert dst.read_text() == "precious existing content"
    encrypt_file(src, dst, passphrase, kdf=fast_kdf, force=True)
    assert dst.read_bytes().startswith(b"SFVAULT")


def test_same_input_and_output_rejected(tmp_path, fast_kdf, passphrase):
    src = tmp_path / "a.txt"
    src.write_text("x")
    with pytest.raises(Exception, match="same file"):
        encrypt_file(src, src, passphrase, kdf=fast_kdf, force=True)
    assert src.read_text() == "x"


def test_no_partial_plaintext_when_late_chunk_is_corrupt(tmp_path, fast_kdf, passphrase):
    """Chunks 1..9 are valid and get decrypted, chunk 10 is corrupt:
    the output path must not exist afterwards and no temp file may remain."""
    src = tmp_path / "big.bin"
```

```
    src.write_bytes(os.urandom(10 * 1024))
    enc = tmp_path / "big.vault"
    encrypt_file(src, enc, passphrase, kdf=fast_kdf, chunk_size=1024)
    b = bytearray(enc.read_bytes())
    b[-TAG_LEN - 3] ^= 0x01                     # corrupt the LAST chunk only
    enc.write_bytes(bytes(b))
    out = tmp_path / "out.bin"
    with pytest.raises(AuthenticationError):
        decrypt_file(enc, out, passphrase)
    assert not out.exists()
    assert _leftovers(tmp_path) == []


def test_failed_decrypt_with_force_keeps_old_output(tmp_path, fast_kdf, passphrase):
    src = tmp_path / "s.txt"
    src.write_text("new secret")
    enc = tmp_path / "s.vault"
    encrypt_file(src, enc, passphrase, kdf=fast_kdf)
    out = tmp_path / "o.txt"
    out.write_text("previous good file")
    with pytest.raises(AuthenticationError):
        decrypt_file(enc, out, "wrong-passphrase", force=True)
    assert out.read_text() == "previous good file"


def test_empty_file_roundtrip(tmp_path, fast_kdf, passphrase):
    src = tmp_path / "empty"
    src.write_bytes(b"")
    encrypt_file(src, tmp_path / "e.vault", passphrase, kdf=fast_kdf)
    decrypt_file(tmp_path / "e.vault", tmp_path / "e.out", passphrase)
    assert (tmp_path / "e.out").read_bytes() == b""


# ---- CLI ------------------------------------------------------------------

def test_cli_has_no_passphrase_argument():
    """Passing a secret on the command line must be impossible."""
    parser = build_parser()
    for cmd in ("encrypt", "decrypt"):
        with pytest.raises(SystemExit):
            parser.parse_args([cmd, "f", "--passphrase", "hunter2-demo"])
    for action in parser._subparsers._group_actions[0].choices["encrypt"]._actions:
        assert "--passphrase" not in action.option_strings


@pytest.mark.parametrize("cipher", ["aes", "chacha"])
def test_cli_roundtrip_both_ciphers(tmp_path, pwfile, cipher, capsys):
    src = tmp_path / "notes.txt"
    src.write_text("synthetic notes " * 1000)
    assert main(["encrypt", str(src), "-c", cipher, "--passphrase-file", pwfile, *FAST]) == EXIT_O
K
    assert main(["info", str(src) + ".vault"]) == EXIT_OK
    info = capsys.readouterr().out
    assert ("ChaCha20" if cipher == "chacha" else "AES-256-GCM") in info
    out = tmp_path / "back.txt"
    assert main(["decrypt", str(src) + ".vault", "-o", str(out), "--passphrase-file", pwfile]) ==
EXIT_OK
    assert out.read_text() == src.read_text()


def test_cli_wrong_passphrase_exit_code(tmp_path, pwfile, capsys):
    src = tmp_path / "a.txt"
    src.write_text("abc")
    main(["encrypt", str(src), "--passphrase-file", pwfile, *FAST])
    bad = tmp_path / "bad.txt"
```

```
    bad.write_text("not-the-passphrase\n")
    bad.chmod(0o600)
    rc = main(["decrypt", str(src) + ".vault", "-o", str(tmp_path / "o"), "--passphrase-file", str(bad)])
    assert rc == EXIT_AUTH
    assert "authentication failed" in capsys.readouterr().err
    assert not (tmp_path / "o").exists()


def test_cli_overwrite_refused_then_forced(tmp_path, pwfile, capsys):
    src = tmp_path / "a.txt"
    src.write_text("abc")
    (tmp_path / "a.txt.vault").write_text("keep me")
    assert main(["encrypt", str(src), "--passphrase-file", pwfile, *FAST]) == EXIT_EXISTS
    assert (tmp_path / "a.txt.vault").read_text() == "keep me"
    assert main(["encrypt", str(src), "--force", "--passphrase-file", pwfile, *FAST]) == EXIT_OK


def test_cli_rejects_non_vault(tmp_path, pwfile):
    f = tmp_path / "plain.vault"
    f.write_bytes(b"hello world, definitely not a vault file" * 3)
    assert main(["decrypt", str(f), "-o", str(tmp_path / "x"), "--passphrase-file", pwfile]) == EXIT_FORMAT


def test_cli_getpass_mismatch(tmp_path, monkeypatch, capsys):
    answers = iter(["first-passphrase-1", "second-passphrase-2"])
    monkeypatch.setattr("getpass.getpass", lambda prompt="": next(answers))
    src = tmp_path / "a.txt"
    src.write_text("x")
    assert main(["encrypt", str(src), *FAST]) != EXIT_OK
    assert "do not match" in capsys.readouterr().err
    assert not (tmp_path / "a.txt.vault").exists()


def test_cli_short_passphrase_rejected(tmp_path, monkeypatch):
    monkeypatch.setattr("getpass.getpass", lambda prompt="": "short")
    src = tmp_path / "a.txt"
    src.write_text("x")
    assert main(["encrypt", str(src), *FAST]) != EXIT_OK
    assert not (tmp_path / "a.txt.vault").exists()


def test_real_terminal_getpass_does_not_echo(tmp_path, passphrase):
    """Drive the CLI through a real pseudo-terminal: the typed passphrase must
    never appear in what the terminal shows."""
    pexpect = pytest.importorskip("pexpect")
    src = tmp_path / "t.txt"
    src.write_text("terminal test")
    cmd = f"{sys.executable} {ROOT / 'vault.py'} encrypt {src} " + " ".join(FAST)
    child = pexpect.spawn(cmd, timeout=30, encoding="utf-8")
    child.expect("Passphrase: ")
    child.sendline(passphrase)
    child.expect("Confirm passphrase: ")
    child.sendline(passphrase)
    child.expect(pexpect.EOF)
    shown = child.before
    child.close()
    assert child.exitstatus == 0, shown
    assert passphrase not in shown
    assert (tmp_path / "t.txt.vault").exists()

```

01-secure-file-vault/tests/test_format.py

```
"""Format layout, header parsing, and a bundled golden test vector."""
import io
from pathlib import Path

from securevault import Cipher, Header, KdfParams, decrypt_stream
from securevault.format import HEADER_LEN, PREAMBLE_LEN

ROOT = Path(__file__).resolve().parents[1]
VECTOR = ROOT / "tests" / "vectors"


def test_layout_matches_spec():
    assert HEADER_LEN == 50 and PREAMBLE_LEN == 82
    h = Header(Cipher.CHACHA20_POLY1305, KdfParams(2, 16384, 2), 4096,
               bytes(range(16)), bytes(range(100, 107)))
    raw = h.pack()
    assert raw[:8] == b"SFVAULT\x00" and raw[8] == 1 and raw[9] == 2 and raw[10] == 1
    assert raw[27:43] == bytes(range(16)) and raw[43:50] == bytes(range(100, 107))
    assert Header.unpack(raw) == h


def test_golden_vectors_still_decrypt():
    """Files written by v1.0.0 must keep decrypting (format stability)."""
    expected = (VECTOR / "plaintext.txt").read_bytes()
    pw = (VECTOR / "passphrase.txt").read_text().strip()
    for name in ("aes.vault", "chacha.vault"):
        out = io.BytesIO()
        decrypt_stream(io.BytesIO((VECTOR / name).read_bytes()), out, pw)
        assert out.getvalue() == expected

```

01-secure-file-vault/tests/test_roundtrip.py

```python
"""Round trips for both ciphers across sizes that hit every chunk boundary."""
import io
import os

import pytest

from securevault import Cipher, decrypt_stream, encrypt_stream, read_header
from securevault.format import HEADER_LEN, PREAMBLE_LEN, TAG_LEN

CHUNK = 1024
SIZES = [0, 1, CHUNK - 1, CHUNK, CHUNK + 1, 3 * CHUNK, 3 * CHUNK + 17, 200_000]


def _enc(data, pw, kdf, cipher, chunk=CHUNK):
    out = io.BytesIO()
    encrypt_stream(io.BytesIO(data), out, pw, cipher=cipher, kdf=kdf, chunk_size=chunk)
    return out.getvalue()


def _dec(blob, pw):
    out = io.BytesIO()
    decrypt_stream(io.BytesIO(blob), out, pw)
    return out.getvalue()


@pytest.mark.parametrize("cipher", list(Cipher))
@pytest.mark.parametrize("size", SIZES)
def test_roundtrip(cipher, size, fast_kdf, passphrase):
    data = os.urandom(size)
    blob = _enc(data, passphrase, fast_kdf, cipher)
    assert _dec(blob, passphrase) == data
    # ciphertext length = preamble + plaintext + one tag per chunk
    chunks = max(1, -(-size // CHUNK))
    assert len(blob) == PREAMBLE_LEN + size + chunks * TAG_LEN


@pytest.mark.parametrize("cipher", list(Cipher))
def test_empty_file_has_one_authenticated_chunk(cipher, fast_kdf, passphrase):
    blob = _enc(b"", passphrase, fast_kdf, cipher)
    assert len(blob) == PREAMBLE_LEN + TAG_LEN
    assert _dec(blob, passphrase) == b""


def test_header_records_cipher(fast_kdf, passphrase):
    for cipher in Cipher:
        blob = _enc(b"abc", passphrase, fast_kdf, cipher)
        header, raw, _ = read_header(io.BytesIO(blob))
        assert header.cipher is cipher and len(raw) == HEADER_LEN


def test_fresh_salt_and_nonce_every_time(fast_kdf, passphrase):
    data = b"same plaintext, same passphrase"
    blobs = [_enc(data, passphrase, fast_kdf, Cipher.AES_256_GCM) for _ in range(20)]
    salts = {read_header(io.BytesIO(b))[0].salt for b in blobs}
    prefixes = {read_header(io.BytesIO(b))[0].nonce_prefix for b in blobs}
    assert len(salts) == 20 and len(prefixes) == 20
    assert len({b[PREAMBLE_LEN:] for b in blobs}) == 20      # ciphertexts differ


def test_unicode_passphrase(fast_kdf):
    pw = "pässwörd-тест-🔑-demo"
    assert _dec(_enc(b"x" * 50, pw, fast_kdf, Cipher.CHACHA20_POLY1305), pw) == b"x" * 50


def test_large_chunk_count_streaming(fast_kdf, passphrase):
```

01-secure-file-vault/tests/test_roundtrip.py (continued)

```
    """5 MiB through 4 KiB chunks = 1280 chunks; memory stays at one chunk."""
    data = os.urandom(5 * 1024 * 1024)
    blob = _enc(data, passphrase, fast_kdf, Cipher.AES_256_GCM, chunk=4096)
    assert _dec(blob, passphrase) == data

```

01-secure-file-vault/tests/test_tamper.py

```
"""Every modification of a vault file must be rejected."""
import io
import os
import struct

import pytest

from securevault import (AuthenticationError, Cipher, FormatError, KdfParams,
                         LimitExceededError, decrypt_stream, encrypt_stream)
from securevault.format import HEADER_LEN, PREAMBLE_LEN, TAG_LEN

CHUNK = 1024
DATA = os.urandom(3 * CHUNK + 100)          # 4 chunks: 3 full + 1 short


@pytest.fixture(params=list(Cipher), ids=lambda c: c.label)
def blob(request, fast_kdf, passphrase):
    out = io.BytesIO()
    encrypt_stream(io.BytesIO(DATA), out, passphrase, cipher=request.param,
                   kdf=fast_kdf, chunk_size=CHUNK)
    return out.getvalue()


def _dec(b, pw, **kw):
    out = io.BytesIO()
    decrypt_stream(io.BytesIO(b), out, pw, **kw)
    return out.getvalue()


def _flip(b, pos, bit=0):
    a = bytearray(b)
    a[pos] ^= 1 << bit
    return bytes(a)


def test_wrong_passphrase(blob, passphrase):
    with pytest.raises(AuthenticationError):
        _dec(blob, passphrase + "x")


def test_wrong_passphrase_fails_before_any_output(blob):
    out = io.BytesIO()
    with pytest.raises(AuthenticationError):
        decrypt_stream(io.BytesIO(blob), out, "definitely-wrong")
    assert out.getvalue() == b""            # commitment check precedes decryption


# Header bytes that are NOT range-checked (salt, nonce prefix) and the commitment
@pytest.mark.parametrize("pos", list(range(27, HEADER_LEN)) + list(range(HEADER_LEN, PREAMBLE_LEN)
))
def test_bitflip_salt_nonce_commitment(blob, passphrase, pos):
    with pytest.raises(AuthenticationError):
        _dec(_flip(blob, pos), passphrase)


@pytest.mark.parametrize("pos", range(0, 27))
def test_bitflip_fixed_header_fields(blob, passphrase, pos):
    """Magic/version/ids/params: rejected as malformed OR as unauthenticated."""
    with pytest.raises((FormatError, AuthenticationError)):
        _dec(_flip(blob, pos), passphrase)


def test_bitflip_every_ciphertext_region(blob, passphrase):
    block = CHUNK + TAG_LEN
    # first byte, middle, and tag region of every chunk
```

```
    positions = []
    for c in range(4):
        start = PREAMBLE_LEN + c * block
        end = min(start + block, len(blob))
        positions += [start, (start + end) // 2, end - TAG_LEN, end - 1]
    for pos in positions:
        with pytest.raises(AuthenticationError):
            _dec(_flip(blob, pos, bit=7), passphrase)


def test_last_tag_byte_flip(blob, passphrase):
    with pytest.raises(AuthenticationError):
        _dec(_flip(blob, len(blob) - 1), passphrase)


@pytest.mark.parametrize("cut", [1, TAG_LEN, 100, CHUNK + TAG_LEN, 100 + TAG_LEN])
def test_truncated_tail(blob, passphrase, cut):
    """Dropping bytes - including exactly the whole final chunk - is detected."""
    with pytest.raises(AuthenticationError):
        _dec(blob[:-cut], passphrase)


def test_truncated_to_whole_chunks_detected(blob, passphrase):
    # Keep exactly 3 full chunks: each is valid, but none carries the final flag.
    b = blob[:PREAMBLE_LEN + 3 * (CHUNK + TAG_LEN)]
    with pytest.raises(AuthenticationError):
        _dec(b, passphrase)


@pytest.mark.parametrize("length", [0, 5, HEADER_LEN - 1])
def test_too_short_is_format_error(blob, passphrase, length):
    with pytest.raises(FormatError):
        _dec(blob[:length], passphrase)


@pytest.mark.parametrize("length", [HEADER_LEN, PREAMBLE_LEN - 1, PREAMBLE_LEN, PREAMBLE_LEN + 5])
def test_header_only_or_no_chunks(blob, passphrase, length):
    with pytest.raises(AuthenticationError):
        _dec(blob[:length], passphrase)


def test_appended_bytes_detected(blob, passphrase):
    for extra in (b"\x00", os.urandom(TAG_LEN), os.urandom(CHUNK + TAG_LEN)):
        with pytest.raises(AuthenticationError):
            _dec(blob + extra, passphrase)


def test_swapped_chunks_detected(blob, passphrase):
    block = CHUNK + TAG_LEN
    c0 = blob[PREAMBLE_LEN:PREAMBLE_LEN + block]
    c1 = blob[PREAMBLE_LEN + block:PREAMBLE_LEN + 2 * block]
    swapped = blob[:PREAMBLE_LEN] + c1 + c0 + blob[PREAMBLE_LEN + 2 * block:]
    with pytest.raises(AuthenticationError):
        _dec(swapped, passphrase)


def test_cipher_id_swap_detected(blob, passphrase):
    """Changing AES<->ChaCha in the header is caught by the commitment HMAC."""
    a = bytearray(blob)
    a[9] = 3 - a[9]
    with pytest.raises(AuthenticationError):
        _dec(bytes(a), passphrase)


def test_not_a_vault_file(passphrase):
```

```
    with pytest.raises(FormatError, match="magic"):
        _dec(b"PK\x03\x04" + os.urandom(200), passphrase)


def test_unknown_version(blob, passphrase):
    a = bytearray(blob)
    a[8] = 2
    with pytest.raises(FormatError, match="version"):
        _dec(bytes(a), passphrase)


# ---- hostile headers: rejected BEFORE running Argon2 -----------------------

def _set_u32(b, off, value):
    a = bytearray(b)
    struct.pack_into(">I", a, off, value)
    return bytes(a)


@pytest.mark.parametrize("off,value,label", [
    (16, 4 * 1024 * 1024, "4 GiB memory"),
    (16, 0xFFFFFFFF, "max u32 memory"),
    (16, 16, "tiny memory"),
    (12, 1_000_000, "1M iterations"),
    (12, 0, "zero iterations"),
    (23, 0xFFFFFFFF, "4 GiB chunk"),
    (23, 1, "1-byte chunk"),
])
def test_hostile_kdf_or_chunk_params_rejected_fast(blob, passphrase, off, value, label):
    import time
    t0 = time.perf_counter()
    with pytest.raises(LimitExceededError):
        _dec(_set_u32(blob, off, value), passphrase)
    assert time.perf_counter() - t0 < 0.5, label


def test_parallelism_bound(blob, passphrase):
    a = bytearray(blob)
    a[20] = 255
    with pytest.raises(LimitExceededError):
        _dec(bytes(a), passphrase)


def test_caller_memory_limit(passphrase):
    out = io.BytesIO()
    encrypt_stream(io.BytesIO(b"data"), out, passphrase,
                   kdf=KdfParams(1, 64 * 1024, 1), chunk_size=CHUNK)
    with pytest.raises(LimitExceededError, match="exceeds"):
        _dec(out.getvalue(), passphrase, max_memory_kib=32 * 1024)
    assert _dec(out.getvalue(), passphrase, max_memory_kib=64 * 1024) == b"data"


def test_encrypt_refuses_out_of_bounds_params(passphrase):
    with pytest.raises(LimitExceededError):
        encrypt_stream(io.BytesIO(b"x"), io.BytesIO(), passphrase,
                       kdf=KdfParams(1, 2 * 1024 * 1024, 1))

```

01-secure-file-vault/.gitignore

```
__pycache__/
*.pyc
.pytest_cache/
.venv/
venv/
*.egg-info/
build/
dist/
*.part
# never commit decrypted output or your own encrypted files by accident
*.decrypted
*.decrypted.*
/my_files/

```

01-secure-file-vault/pyproject.toml

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "securevault"
version = "1.0.0"
description = "Secure file vault: Argon2id + AES-256-GCM / ChaCha20-Poly1305 with an authenticated
, versioned format"
requires-python = ">=3.9"
dependencies = ["cryptography>=42.0", "argon2-cffi>=23.1"]

[project.optional-dependencies]
ui = ["streamlit>=1.43"]
test = ["pytest>=8.0", "pexpect>=4.9"]

[project.scripts]
vault = "securevault.cli:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
testpaths = ["tests"]

```

```python
"""Measure KDF cost and encryption throughput; write outputs/metrics.json."""
import io
import json
import os
import platform
import subprocess
import sys
import time
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(ROOT / "src"))
from securevault import Cipher, KdfParams, decrypt_stream, encrypt_stream  # noqa: E402
from securevault.kdf import derive_keys  # noqa: E402


def best_of(fn, n=3):
    times = []
    for _ in range(n):
        t0 = time.perf_counter(); fn(); times.append(time.perf_counter() - t0)
    return min(times)


def main():
    metrics = {"machine": {"python": platform.python_version(), "cpus": os.cpu_count(),
                           "platform": platform.platform()}}
    salt = os.urandom(16)
    kdf_rows = {}
    for label, p in {"t1_m8MiB_p1 (test minimum)": KdfParams(1, 8 * 1024, 1),
                     "t3_m64MiB_p4 (default)": KdfParams(3, 64 * 1024, 4),
                     "t3_m256MiB_p4 (paranoid)": KdfParams(3, 256 * 1024, 4)}.items():
        kdf_rows[label] = round(best_of(lambda: derive_keys("benchmark-passphrase", salt, p)) * 1000, 1)
    metrics["argon2id_ms"] = kdf_rows

    fast = KdfParams(1, 8 * 1024, 1)
    data = os.urandom(64 * 1024 * 1024)
    thr = {}
    for c in Cipher:
        enc = io.BytesIO()
        te = best_of(lambda: (enc.seek(0), enc.truncate(), encrypt_stream(io.BytesIO(data), enc, "benchmark-passphrase", cipher=c, kdf=fast)), 2)
        blob = enc.getvalue()
        td = best_of(lambda: decrypt_stream(io.BytesIO(blob), io.BytesIO(), "benchmark-passphrase"), 2)
        thr[c.label] = {"encrypt_MiB_s": round(64 / te, 1), "decrypt_MiB_s": round(64 / td, 1),
                        "overhead_bytes_64MiB": len(blob) - len(data)}
    metrics["throughput_64MiB_chunk64KiB"] = thr

    # test-suite result (run pytest and record counts)
    p = subprocess.run([sys.executable, "-m", "pytest", "-q", "-p", "no:cacheprovider"], cwd=ROOT,
                       capture_output=True, text=True, timeout=600)
    last = [l for l in p.stdout.splitlines() if " passed" in l or " failed" in l]
    metrics["pytest_summary"] = last[-1] if last else p.stdout[-200:]
    metrics["pytest_exit_code"] = p.returncode
    (ROOT / "outputs").mkdir(exist_ok=True)
    (ROOT / "outputs" / "metrics.json").write_text(json.dumps(metrics, indent=2))
    print(json.dumps(metrics, indent=2))


if __name__ == "__main__":
    main()

```

01-secure-file-vault/scripts/demo_cli.py

```
"""Run the real CLI end to end and record its exact output to samples/cli_output.txt.

Uses the synthetic sample and the demo passphrase file (both generated by
make_sample.py). One step drives the interactive getpass prompt through a real
pseudo-terminal so the transcript shows what a user sees.
"""
import os
import shutil
import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
S = ROOT / "samples"
WORK = ROOT / "samples" / "demo_run"
PW = "samples/demo-passphrase.txt"
VAULT = [sys.executable, "vault.py"]
log = []


def run(args, label):
    shown = "$ vault " + " ".join(args)
    p = subprocess.run(VAULT + args, cwd=ROOT, capture_output=True, text=True, timeout=120)
    log.append(f"# {label}\n{shown}\n{p.stdout}{p.stderr}[exit code {p.returncode}]\n")
    return p.returncode


def main():
    if WORK.exists():
        shutil.rmtree(WORK)
    WORK.mkdir(parents=True)
    src = "samples/synthetic_customers.csv"
    d = "samples/demo_run"
    run(["encrypt", src, "-o", f"{d}/customers.aes.vault", "--passphrase-file", PW], "1. encrypt with AES-256-GCM (default Argon2id t=3, 64 MiB, p=4)")
    run(["encrypt", src, "-o", f"{d}/customers.chacha.vault", "-c", "chacha", "--passphrase-file", PW], "2. encrypt with ChaCha20-Poly1305")
    run(["info", f"{d}/customers.aes.vault"], "3. inspect the header - no passphrase needed (this is the metadata anyone can see)")
    run(["decrypt", f"{d}/customers.aes.vault", "-o", f"{d}/customers.decrypted.csv", "--passphrase-file", PW], "4. decrypt")
    same = (ROOT / src).read_bytes() == (ROOT / d / "customers.decrypted.csv").read_bytes()
    log.append(f"# 4b. byte-for-byte identical to the original: {same}\n")
    run(["decrypt", f"{d}/customers.chacha.vault", "-o", f"{d}/customers.decrypted.csv", "--passphrase-file", PW], "5. decrypt again to the same path - refused without --force")
    wrong = WORK / "wrong-pass.txt"
    wrong.write_text("not-the-right-passphrase\n"); wrong.chmod(0o600)
    run(["decrypt", f"{d}/customers.aes.vault", "-o", f"{d}/wrong.csv", "--passphrase-file", f"{d}/wrong-pass.txt"], "6. wrong passphrase")
    log.append(f"# 6b. output file created after failure: {(WORK / 'wrong.csv').exists()}\n")
    blob = bytearray((WORK / "customers.aes.vault").read_bytes())
    blob[len(blob) // 2] ^= 0x01
    (WORK / "tampered.vault").write_bytes(bytes(blob))
    run(["decrypt", f"{d}/tampered.vault", "-o", f"{d}/tampered.csv", "--passphrase-file", PW], "7. one bit flipped in the ciphertext")
    good = (WORK / "customers.aes.vault").read_bytes()
    (WORK / "truncated.vault").write_bytes(good[:-40])
    run(["decrypt", f"{d}/truncated.vault", "-o", f"{d}/truncated.csv", "--passphrase-file", PW], "8. last 40 bytes cut off")
    hostile = bytearray(good)
    hostile[16:20] = (4 * 1024 * 1024).to_bytes(4, "big")      # ask for 4 GiB of Argon2 memory
    (WORK / "hostile.vault").write_bytes(bytes(hostile))
    run(["decrypt", f"{d}/hostile.vault", "-o", f"{d}/hostile.csv", "--passphrase-file", PW], "9. hostile header asking for 4 GiB of Argon2 memory - rejected before key derivation")
    (WORK / "empty.txt").write_bytes(b"")
```

```
    run(["encrypt", f"{d}/empty.txt", "--passphrase-file", PW], "10. empty file")
    run(["decrypt", f"{d}/empty.txt.vault", "-o", f"{d}/empty.out", "--passphrase-file", PW], "10b. decrypt empty file")

    # Interactive prompt through a real terminal (passphrase typed, never echoed)
    try:
        import pexpect
        child = pexpect.spawn(f"{sys.executable} vault.py encrypt {src} -o {d}/interactive.vault",
                              cwd=str(ROOT), encoding="utf-8", timeout=60)
        child.expect("Passphrase: "); child.sendline("demo-only-correct-horse-battery")
        child.expect("Confirm passphrase: "); child.sendline("demo-only-correct-horse-battery")
        child.expect(pexpect.EOF)
        transcript = "Passphrase: \nConfirm passphrase: " + child.before.replace("\r", "")
        child.close()
        log.append(f"# 11. interactive getpass in a real terminal (typed characters are not echoed)\n"
                   f"$ vault encrypt {src} -o {d}/interactive.vault\n{transcript}[exit code {child.exitstatus}]\n")
    except ImportError:
        log.append("# 11. (pexpect not installed - interactive demo skipped)\n")

    text = "\n".join(log)
    (S / "cli_output.txt").write_text(text)
    print(text)
    # keep only small artefacts; delete decrypted plaintext copies from the demo dir
    for p in WORK.iterdir():
        if not p.name.endswith(".vault"):
            p.unlink()


if __name__ == "__main__":
    main()

```

01-secure-file-vault/scripts/make_sample.py

```python
"""Generate SYNTHETIC sample files (no real personal data, no real passwords)."""
import csv
import random
from pathlib import Path

ROOT = Path(__file__).resolve().parents[1]
OUT = ROOT / "samples"


def main(seed: int = 7) -> None:
    rng = random.Random(seed)
    OUT.mkdir(exist_ok=True)
    first = ["Asha", "Ravi", "Meera", "Kiran", "Arjun", "Divya", "Sanjay", "Lakshmi"]
    last = ["Rao", "Reddy", "Iyer", "Sharma", "Naidu", "Kapoor"]
    with open(OUT / "synthetic_customers.csv", "w", newline="") as fh:
        w = csv.writer(fh)
        w.writerow(["customer_id", "name", "city", "account_balance_inr", "note"])
        for i in range(1, 201):
            w.writerow([f"C{i:04d}", f"{rng.choice(first)} {rng.choice(last)}",
                        rng.choice(["Vijayawada", "Hyderabad", "Chennai", "Pune"]),
                        rng.randint(1_000, 900_000), "SYNTHETIC RECORD - generated for demo"])
    (OUT / "demo-passphrase.txt").write_text("demo-only-correct-horse-battery\n")
    (OUT / "demo-passphrase.txt").chmod(0o600)
    print(f"wrote {OUT / 'synthetic_customers.csv'} and demo-passphrase.txt (demo only)")


if __name__ == "__main__":
    main()

```

01-secure-file-vault/vault.py

```
#!/usr/bin/env python3
"""Run the CLI without installing:  python vault.py encrypt secret.pdf"""
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent / "src"))
from securevault.cli import main  # noqa: E402

if __name__ == "__main__":
    sys.exit(main())

```